



AGENTIC REINFORCEMENT LEARNING

智能体强化学习（Agentic RL）

从「会回答」到「会完成任务」

把 LLM 当作能感知、规划、调用工具、记忆、反思并与环境多轮交互的 **Agent** 来训练，而不是只当作一次性生成答案的文本模型来优化。

技术分享 · 综述 + 算法/系统深度 + DeepEnlighten 复现实践 · 截至 2026-06

分享目标



为什么会从「单轮偏好优化」走向「多步交互能力优化」

RLHF → RLVR → Agentic RL 的动因



Agentic RL 与传统 RL / LLM RL / RLHF / 推理模型是什么关系

范式坐标与形式化区分



Agentic RL 增强哪些能力，已在哪些任务场景发挥作用

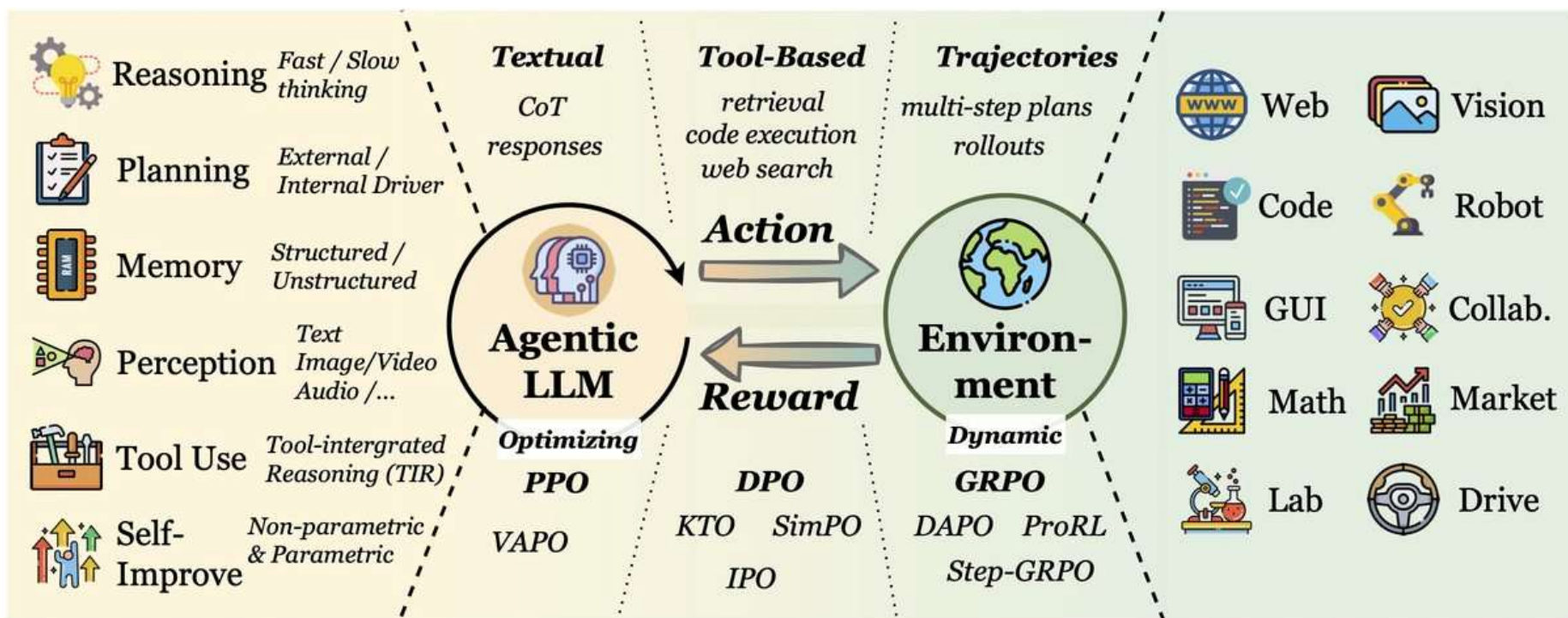
六大能力 × 六类任务域



R1-Zero 的启发如何迁移到更小、可复现的实践

DeepEnlighten: Social IQa + 纯 RL 的泛化观察

智能体在「环境循环」中被优化



图源: The Landscape of Agentic RL for LLMs (Survey), Fig.3 — 左: 六大能力; 中: 动作粒度 (文本/ 工具/ 轨迹) 与优化算法; 右: 环境域。

读法: 模型据观测产生 **Action** → 环境执行并返回 **Reward** → 信号回流优化策略。优化算法随交互粒度递进: PPO/VAPO (文本) → DPO 系 (工具) → GRPO/DAPO/Step-GRPO (轨迹)。

1

PART 1

为什么需要 Agentic RL

从「会回答」到「会推理」再到「会完成任务」

会回答 → 会推理 → 会完成任务



RLHF

阶段一 · 会回答

- 三步流水线：SFT → 训练奖励模型(RM) → PPO 优化
- 奖励来自人类偏好；KL 约束防偏离
- 面向单轮 / 短上下文，本质是一次性输出



RLVR

阶段二 · 会推理

- 奖励来自可验证结果（答案对错 / 测试通过）
- 无需昂贵人工偏好；结果奖励间接塑造过程
- R1-Zero：纯 RL 涌现反思、验证、长 CoT



Agentic RL

阶段三 · 会完成任务

- 多轮、与环境交互、调用工具
- 每步行动改变环境、暴露新信息
- 成败取决于整条轨迹，而非单个 token

关键转折：奖励信号从「人类觉得哪个好」→「答案是否客观正确」→「任务是否真的完成」。

为什么「会回答」还不够



做研究

搜索 → 打开网页 → 比较来源 → 提炼 → 形成报告



写代码

读仓库 → 定位 bug → 改文件 → 跑测试 → 据报错续修



操作 GUI

观察界面 → 点击 → 填表单 → 等页面变化 → 决定下一步



用工具

判断何时调用搜索 / 计算器 / 数据库 / 代码解释器



多智能体协作

分工 → 沟通 → 汇总 → 避免重复劳动

三个共同点

- ① 必须在环境中行动，而非只生成最终答案
- ② 每步行动改变后续可见信息
- ③ 好坏取决于整条轨迹，而非某句话

⇒ 把 LLM 从「回答器」训练成「能完成任务的策略」

RL 后训练的演进脉络



RLHF/DPO 塑造性格 (有用、安全、好聊) | RLVR/Agentic RL 锻造能力 (会推理、会用工具、能完成成长任务)

经典 RLHF · 偏好优化 · RLVR/Agentic RL

维度	经典 RLHF	偏好优化 (DPO 系)	RLVR / Agentic RL
代表	InstructGPT	DPO · SimPO · KTO	DeepSeek-R1 · Search-R1
奖励来源	学习型 RM (人类偏好)	偏好对 (隐式奖励)	可验证器 / 规则 / 结果
是否需 RM	需要	不需要 (隐式)	不需要 (用验证器)
是否需 RL 循环	需要 (PPO)	不需要	需要 (GRPO/PPO)
典型任务	主观对齐、对话	主观对齐、对话	数学/代码/推理、工具、智能体
信号客观性	主观、易被 hack	主观	客观、抗 hack (可验证时)

一句话：前者塑造性格，后者锻造能力，两条线在前沿模型中并存互补。

2

PART 2

RL 基础与算法谱系

MDP · 轨迹 · 策略 · 奖励; REINFORCE → PPO → GRPO → DAPO/GSPO

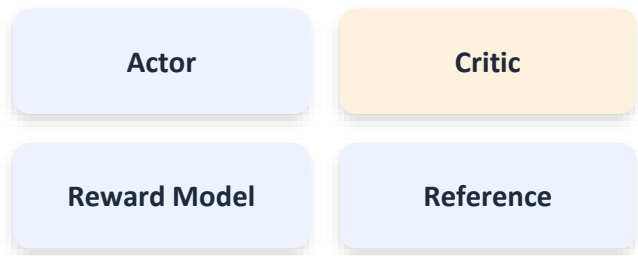
强化学习基础

MDP	$M = (S, A, P, R, \gamma)$	状态/动作/转移/奖励/折扣。LLM 里：状态=prompt+历史+工具返回；动作=生成文本 / 调工具 / 点击
轨迹 τ	$(s_0, a_0, r_0, \dots, s_T, a_T, r_T)$	单轮任务可能只有 prompt+answer；Agentic 可达几十~上百步。难点：把末端奖励归因到前面每步
策略 $\pi_\theta(a s)$	= 模型的 next-token 分布	动作空间极大、探索成本高；同一高层动作有多种文本表达 → 信用分配更难
奖励 R	结果 / 过程 / 格式 / 成本 / 安全	RL 的方向盘。设计得坏 → 模型学会作弊、投机、格式 hack
Return / Value / Advantage	$A_t = G_t - V(s_t)$	Advantage>0 → 提高该动作概率；<0 → 降低。是所有策略梯度的核心

PPO → GRPO 及其变体

PPO

裁剪式更新 + 独立 critic



四个同规模模型 → 显存/算力开销大

GRPO

组内相对优势，去掉 critic



同题采样一组 → 组内归一化得优势

$$\hat{A}_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

2025 进阶变体：都在「防崩」

- **DAPO**: Clip-Higher / 动态采样 / token级损失 / 超长惩罚
- **GSPO** (Qwen3): 序列级重要性比, 修正 token 级误用, 稳 MoE
- **Dr.GRPO** / **REINFORCE++** / **VAPO**: 去偏 / 全局归一化 / 价值改进

GRPO 是主干; 变体回答「长序列+大模型+稀疏奖励下如何不崩」

目标函数

$$J_{GRPO}(\theta) = E[(1/G) \sum_i (1/|o_i|) \sum_t \min(r_{it}(\theta) \cdot \hat{A}_i, \text{clip}(r_{it}(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_i) - \beta \cdot D_{KL}(\pi_\theta \| \pi_{ref})]$$

GSPO 改用序列级比值: $s_i(\theta) = (\pi_\theta(o_i|q) / \pi_{old}(o_i|q))^{(1/|o_i|)}$ ← 长度归一化, 降方差、稳 MoE

算法对比

算法	需 critic	优势粒度	关键卖点	主要风险
PPO	是	token (GAE)	经典稳定	critic 开销大、长序列偏差
REINFORCE	否	轨迹回报	极简、易复现	方差大、不稳
GRPO	否	组相对 / token	省显存、可扩展	熵崩溃、估计偏差
REINFORCE++	否	全局归一化	稳定、简单	仍属单步范式
DAPO	否	token 级损失	长链 RL 工程化、维持探索	技巧需调参
GSPO	否	序列级比值	稳定、利好 MoE	信息粒度较粗
VAPO	是	token (改进 GAE)	长序列、稀疏奖励	实现复杂

GSPO 的洞见：token 级重要性采样在理论上「被误用」——单样本高方差噪声随长度累积、被裁剪放大 → 不可逆崩溃。

单步 MDP → 长程 POMDP

维度	LLM-RL (单步)	Agentic RL (多步)
决策结构	退化单步 MDP	时序扩展 POMDP
动作	整段文本	文本动作 + 操作动作
环境转移	确定性	随机、动态
时间跨度	1 步	100+ 轮 / 10万-100万 token
奖励	末端单一标量	稀疏、延迟、可含过程奖励
核心难点	长链生成稳定性	信用分配 / 探索 / 环境稳定 / rollout

结构化动作示例

```
{
  "tool": "web_search",
  "query": "Agentic RL survey"
}
```

```
{
  "action": "click",
  "target": "Submit"
}
```

好处：环境易执行/验证、reward 易归因。代价：要学会填对参数、理解返回、失败后恢复。

这张表是整个领域的「世界观」：难点根源在于从单步变为长程随机 POMDP。

3

PART 3

多轮难题与六大能力

信用分配 · Echo Trap · 熵崩溃；推理 / 工具 / 深度研究等能力的训练机制

稀疏奖励 + 随机环境带来的三道坎



时序信用分配

轨迹横跨 100+ 轮，奖励只在最后给出。

- 轨迹级：整条共享一个优势，长程信号极弱
- 轮次级 / 步级：更细粒度 (VinePPO、GiGPO、Turn-PPO)
- turn-level reward 让工具调用成功率→100%



Echo Trap 与不稳定

RAGEN/StarPO：奖励方差骤降、梯度尖峰。

- 模型陷入重复浅层策略 / 幻觉式思考
- StarPO-S：轨迹过滤 + critic + 梯度稳定
- 需多样初始状态、中等粒度、更频繁采样

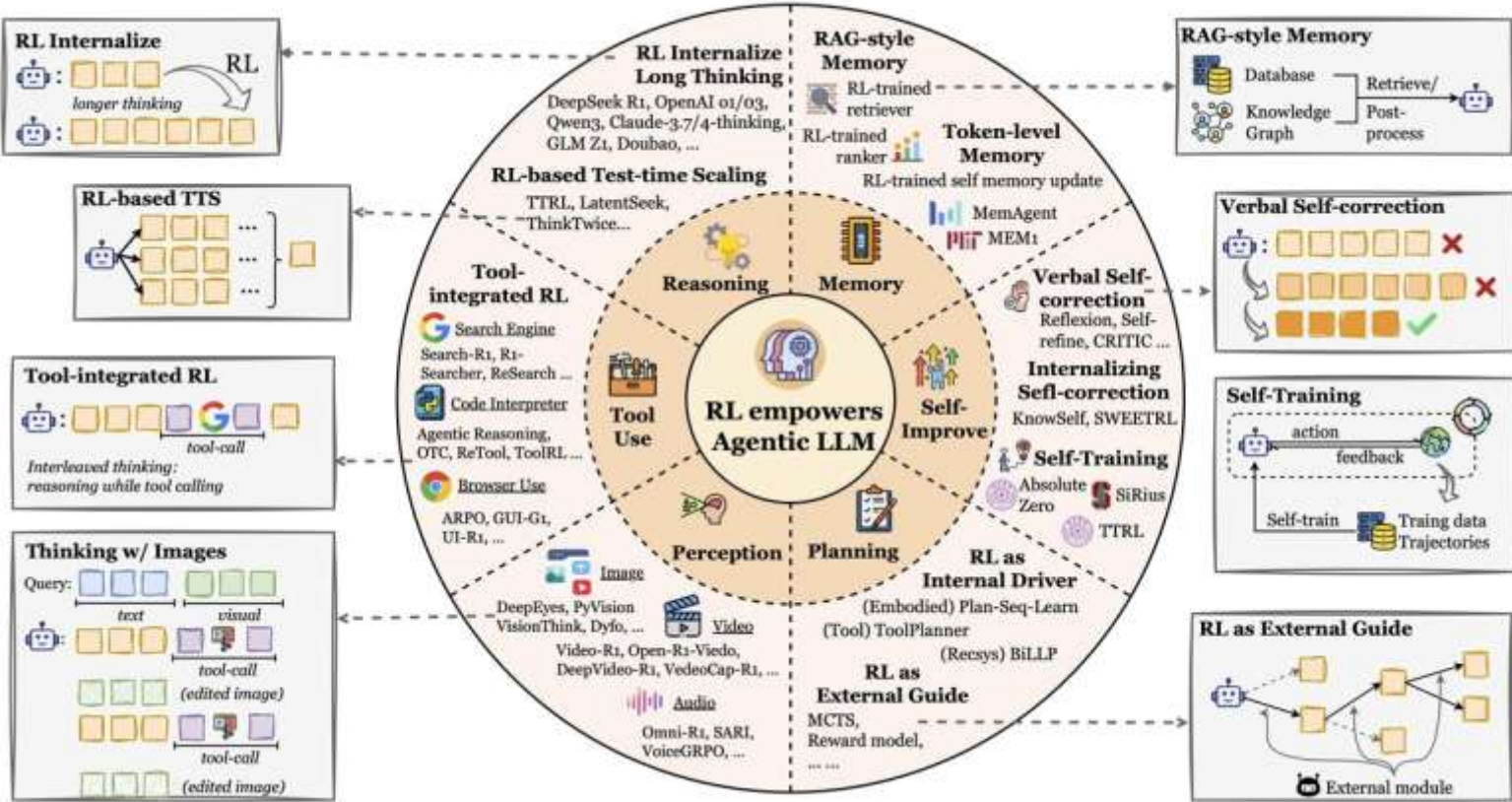


探索与熵崩溃

策略熵过早单调下降，输出迅速趋同。

- 少量高熵「分叉 token」主导推理多样性
- 缓解：熵奖励 / Clip-Higher / AEPO
- 反直觉：弱/随机奖励也能涨分 → 分数≠能力

六大能力版图



RL 在做什么

- **Planning** 拆解目标、定执行序
- **Tool Use** 何时/调哪个/怎么用
- **Memory** 何时写入/检索/更新
- **Reasoning** 长 CoT、验证、回溯
- **Self-Improve** 从成败轨迹中学
- **Perception** 主动感知、改变观测

图源：Survey Fig.5 — RL 把静态启发式模块转化为自适应、鲁棒的智能体行为。

基座 + GRPO + 两个规则奖励

① 固定输出格式 (system prompt)

```
<think> 推理过程 </think>
<answer> 最终答案 </answer>
```

② 奖励 = 准确率 + 格式

- **准确率**: 验证器判 <answer> (数学用 SymPy, 代码跑测试)
- **格式**: 是否正确使用标签, 便于抽取答案
- **③ GRPO**: 每题采样一组($G=16$), 组内归一化更新

实例 · Countdown 数字游戏

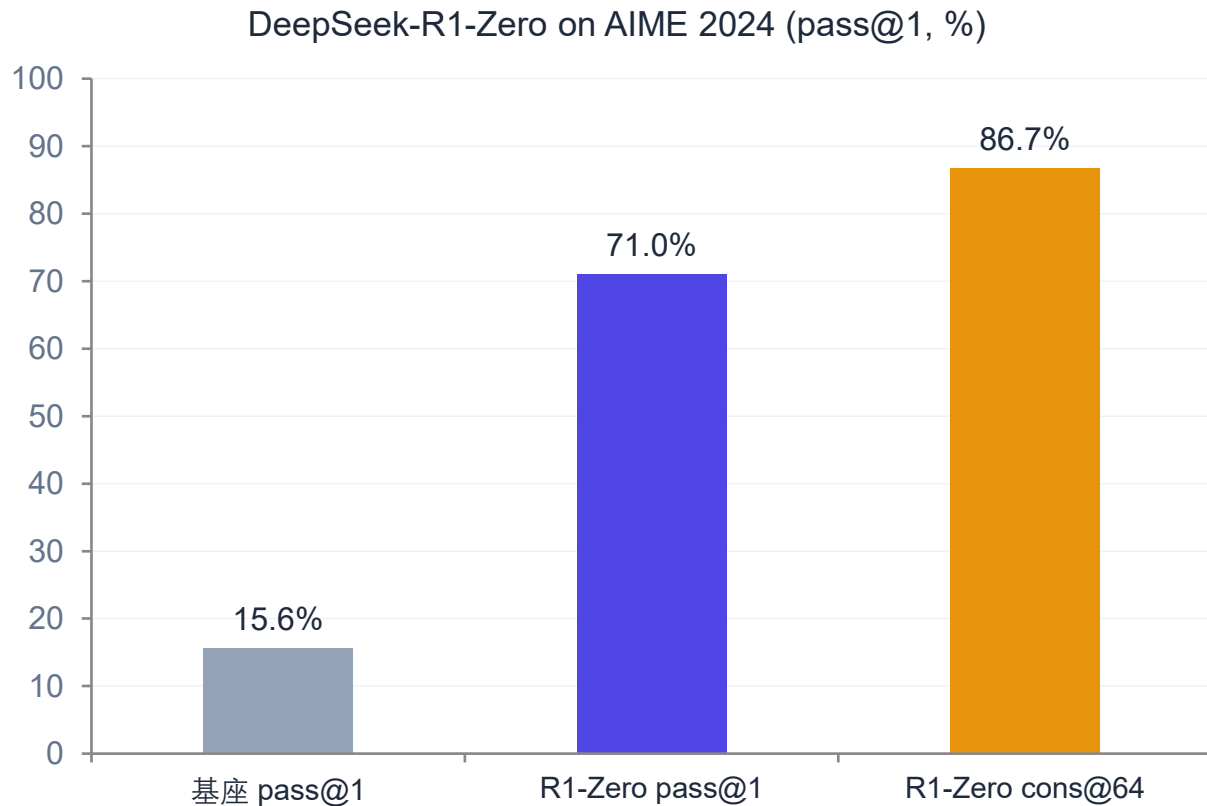
给定 [3, 7, 11, 25], 目标 90; 每个数恰用一次、 $+-\times\div$ 凑出目标。

采样轨迹 (示意)

```
<think> 25×3=75, 75+11=86,
      86+7=93 X ; 试 (7-3)=4,
      4×25=100, 100-11=89 X ...
      (11-7)×25=100? 重试 →
      3×25=75, 75+11=? </think>
<answer> (25-11)×7-3×? </answer>
```

三层奖励即「课程」: 格式 → 有效性 (每数用一次) → 准确率 (结果=目标)。纯规则、纯结果, 小模型几百步即学会系统试探+自检。

只靠「答案对不对」一个信号涌现出的行为



自发涌现的三种行为

- 对难题自动延长思维链
- 走不通时回退、重新评估
- 拟人化「等一下，让我重新想想」式自我纠错

训练动态：回答长度随训练自发增长 → 模型自己学会「想得更久」，下游性能同步上升（测试时扩展被内生出来）。

⚠ 长 CoT 不是目标本身——目标是可验证任务的成功率。

Search-R1：边推理边检索

提示词模板 (Search-R1)

prompt template

回答问题。每次拿到新信息后先在
<think></think> 内推理。
若知识不足，用 <search>查询</search>
调用搜索；结果返回在
<information>...</information> 之间。
可搜索任意多次；无需更多知识则
直接在 <answer>答案</answer> 作答。

一条多轮轨迹 (示意)

rollout trajectory

```
<think>需要查 X</think>
<search>X 是什么</search>
<information>...检索结果...</information>
<think>结合结果，还需查 Y</think>
<search>...</search><information>...
<think>够了</think>
<answer>...最终答案...</answer>
```

奖励： Search-R1 用纯结果奖励（答案 EM）即可；Qwen2.5-7B 比 RAG 基线平均 +26%。早期常依赖 ReAct (Thought→Action→Observation) 提示，RL 进一步学会「何时该搜、何时换策略、减少无效调用」。

检索 token 掩码 + 分段奖励设计

机制 A · 检索/观察 token 掩码（极关键）



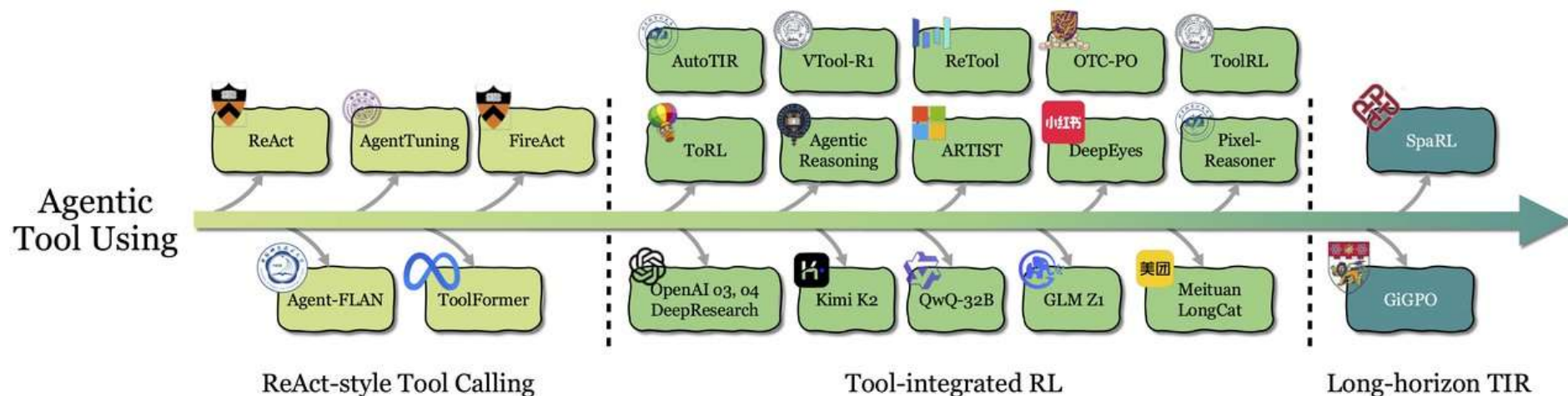
只对模型自己生成的 token 求梯度；工具返回的 token 掩码（不参与损失）。否则模型会去「模仿检索结果分布」→ 不稳定、学会照抄。消融证实：加掩码更稳、更好。

机制 B · ToolRL 分段奖励设计

```
R_final = R_format + R_correct
```

- 四维度消融：奖励类型 / 尺度 / 粒度 / 时序动态
- 细粒度分段（对工具名、参数分别给分）比粗粒度「答案匹配」学得更稳
- **反直觉**：从原始 instruct 直接冷启动 RL，泛化优于先 SFT 再 RL（SFT=记忆，损害探索）
- **效果**：比基座 +17%、比 SFT +15%；涌现主动验证、拒绝不当调用等元认知

从 ReAct 提示 → 工具集成 RL → 长程 TIR



图源: Survey Fig.4 — Agentic Tool Using 的演化: ReAct-style → Tool-integrated RL → Long-horizon TIR。

实例 · ReTool: 在推理中实时跑代码 (字节 Seed)

- **两阶段:** ① 冷启动 SFT 教代码调用格式 → ② RL 在沙箱代码解释器多轮 rollout, 结果奖励教「何时调代码、如何从报错中自我纠正」
- **效果:** AIME 显著超纯文本 RL 基线; RL 后回答长度比训练前缩短 ~40% —— 学会「该用工具时用, 不再啰嗦硬推」

长程 + 真实开放环境 + 多源综合

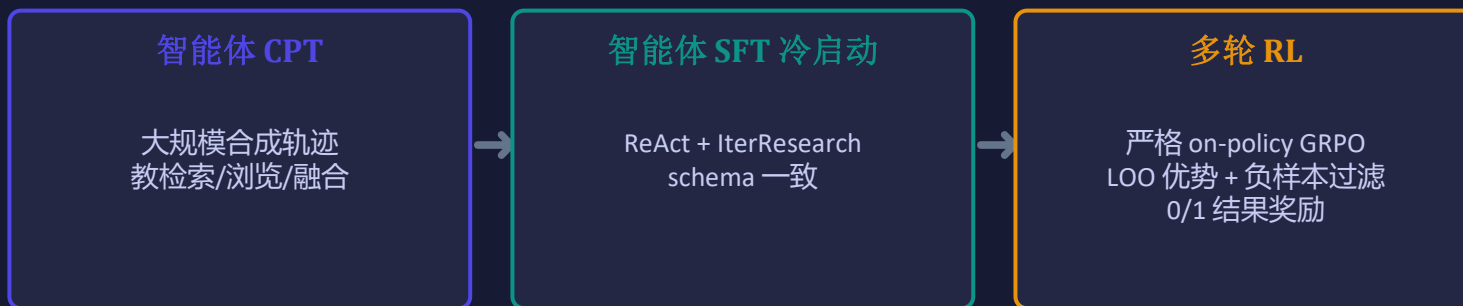
为什么特别难

- 真实环境噪声不可控：限流、反爬、网页结构千差万别、延迟
- 超长程、稀疏奖励：几十轮、十万级 token，奖励只在最后
- 训练数据稀缺：高质量「研究轨迹」几乎无法人工标注

DeepResearcher (GAIR)：首个在真实网络端到端 RL

用 GRPO 直接与真实搜索引擎交互；涌现计划、跨源交叉验证、自我反思、找不到时保持诚实。比提示工程基线 +28.9 分。

Tongyi DeepResearch (30.5B/激活3.3B)：工业级三段式



配套全自动数据合成 (无需人工标注) + Heavy Mode 测试时扩展 → HLE 38.3%，多基准开源 SOTA。

递进：推理（脑内）→ + 工具反馈/掩码 = 工具使用 → + 真实环境/数据合成 = 深度研究。同一套 RL 机理在交互复杂度上的逐级放大。

配方 · 奖励 · 必备技巧 · 标志数字

能力	最小配方	奖励设计	必备技巧	标志性数字
推理	基座 + GRPO + 规则奖励	准确率 + 格式	组内归一化、长度自增	R1-Zero AIME 15.6%→71.0%
工具使用	+ 交错模板 + 沙箱/检索	R_format + R_correct（分段）	观察 token 掩码、冷启动 RL	ReTool 超基线、回答 -40%
深度研究	+ 数据合成 + 真实环境	0/1 结果 + rubric/引用	on-policy 稳定化、上下文管理	Tongyi DR HLE 38.3%（ Heavy）

交互复杂度逐级放大

推理（单轮、内部 CoT）
└─ + 外部工具反馈、观察掩码 → 工具使用（多轮、单/少工具）
 └─ + 真实开放环境、长程、多源综合、数据合成 → 深度研究（超长程智能体）

4

PART 4

任务场景与工程核心

六类任务域 · 环境比算法更重要 · reward hacking · 开放挑战

六类任务域与代表系统



搜索/研究

跨源验证、引用、报告

DeepResearcher · Tongyi DR · WebDancer



代码

可执行 verifier 最可靠

SWE-RL · DeepSWE · CWM



数学

答案可验证、数据易构造

R1 · Kimi k1.5 · 形式化(Lean)



GUI / 计算机使用

屏幕/DOM/控件, 副作用风险

UI-TARS-2 (OSWorld 47.5)



视觉 / 具身

主动感知、低层控制

VLM agent · embodied



多智能体

credit assignment 更复杂

planner/coder/critic 协作

环境类型 · 训练框架 · 系统瓶颈

环境类型	优点	缺点
静态问答数据集	易复现、reward 明确	不能训多步行动
工具调用模拟	可训工具选择/参数	与真实工具有差异
代码执行环境	reward 可靠、有价值	sandbox/依赖/安全复杂
Web / GUI	接近真实场景	不稳定、评估难、有副作用
游戏 / 模拟器	可控、可重复	与真实任务差距大

主流训练框架

- verl / HybridFlow (字节) 最成熟, AgentLoop 多轮 rollout
- OpenRLHF 最早流行, REINFORCE++ 发源
- AReal / ROLL / NeMo-RL / SkyRL / rLLM 异步与多轮
- RAGEN / Agent Lightning 多轮环境 / 工作流接入

瓶颈常不在反向传播，而在 rollout 与环境执行：长多轮 episode 的长尾、同步屏障、策略陈旧 → 异步化、P/D 分离、KV-cache 交换是工程主线。

Reward Hacking 陷阱与训练闭环

Reward Design 四类陷阱

- **Reward hacking**: 改测试让自己通过、利用漏洞
- **过度优化格式**: 套模板拿格式分, 质量没提升
- **成本信号失衡**: 过罚→不敢用工具; 不罚→无限搜索/过度思考
- **evaluator bias**: RM 偏好冗长 → 模型写长而空的推理
- **原则**: 奖励尽量贴近真实任务成功, 少量辅助信号约束格式/安全/成本

最小训练闭环 (8 件套)

- 1 Base model (初始策略)
- 2 Task dataset (任务/环境种子)
- 3 Environment (执行动作、返回观测)
- 4 Reward function (结果/过程打分)
- 5 Rollout engine (采样多条轨迹)
- 6 RL trainer (据 reward 更新)
- 7 Evaluator (固定验证集, 看是否真变好)
- 8 Monitor (reward/长度/成功率/KL/样例)

新团队建议: 从可验证、短轨迹、低副作用任务起步 (数学/逻辑/单测/受控工具)。

四类工程挑战 + 一个核心争论



Trustworthiness

能行动 → 错误直接改变外部状态: prompt injection、泄漏、越权



Environment realism

太简单 → 学到 benchmark 策略; 太真实 → 成本/安全难控



Scaling

更长轨迹、更复杂环境、更高 rollout 成本与更多 verifier



Deployment guardrails

权限分层、工具白名单、审计、human-in-the-loop

机制争论: RL 改变了什么?

- **放大器观点:** RLVR 重塑输出分布, 把质量集中到「本就可达」的路径; 大 k 的 pass@k 基座能追平
- **新知识观点:** RL 能装新计算、改善 OOD、诱导基座本无的行为
- **工程结论:** 看 OOD 泛化, 而非只看训练 reward

延伸: Agentic RL (真实环境、外部信息、工具) 正被寄望为突破基座能力上限的路径之一。

5

PART 5

DeepEnlighten 复现实践

Social IQa + 纯 RL：从 EQ 到 IQ 的泛化观察

项目定位 · 实验组件 · 奖励设计

核心问题：不用 SFT，只用规则奖励 + RL 在 Social IQa 上训小模型，能否学到更好的社会推理？这种训练会泛化到数学等 OOD 任务吗？

实验组件

RL 框架	verl
RL 算法	REINFORCE++（轻量、可解释、易调试）
训练数据	Social IQa（日常社会/情绪推理）
基座模型	Qwen2.5-3B · Llama3.2-3B
评估	DeepSeek-Math 等数学 benchmark（OOD）

奖励设计（规则，无 reward model）

```
reward rules

Format reward  合法 +2 / 非法 -2
Answer reward  正确 +2
                错误 -2 / 无效 -3
Language consistency 未使用
```

取舍：先把奖励做简单、可解释、可复现——失败时易定位是数据/格式/算法/采样的问题。代价：信号粗，无法评中间推理质量。

Social IQa 长这样；流程沿用 R1-Zero 精神

Social IQa 训练样本（示意）

data format

Context: Alex 凑近 Jordan 低声说了
一件事。

Question: Alex 为什么这么做?

- (A) 想让别人都听见
- (B) 避免被旁人听到 ← 正确
- (C) 因为他在生气

模型输出: <think>...社会常识推断...</think>
<answer>B</answer>

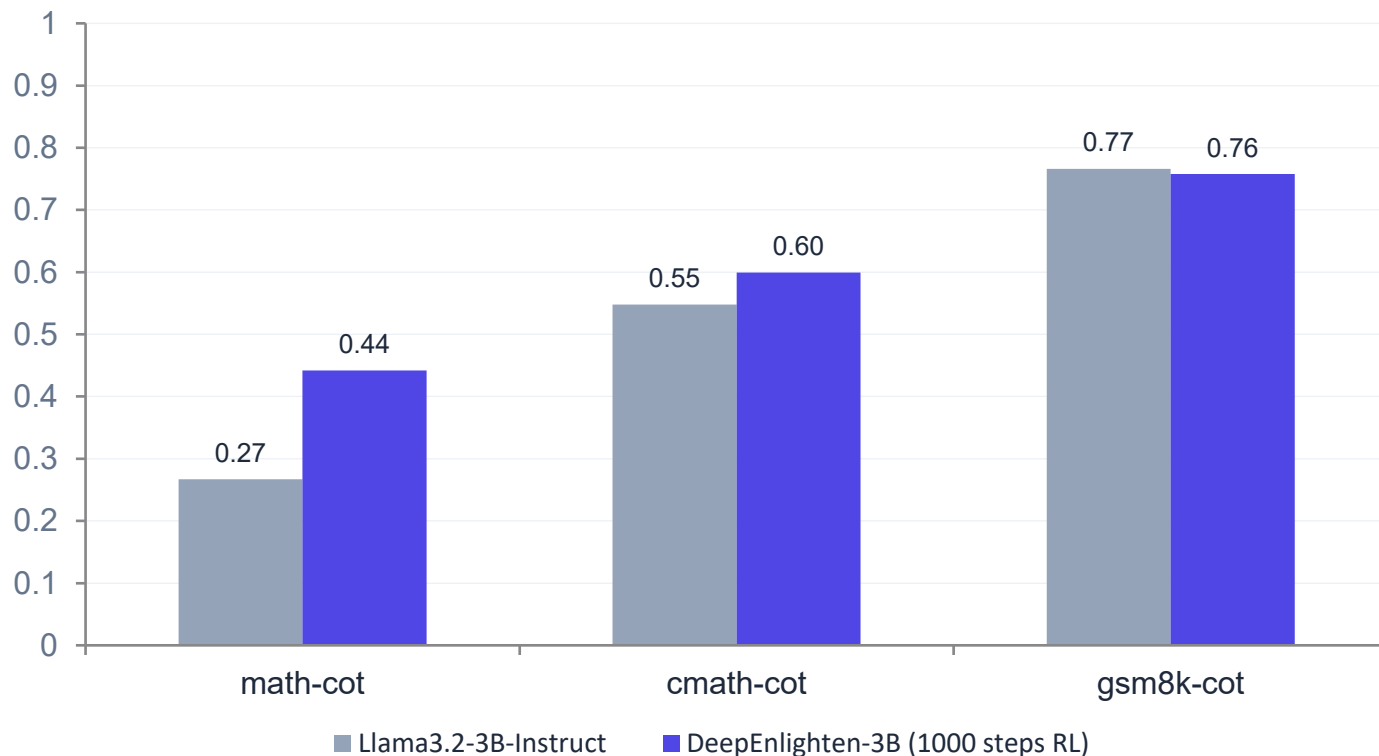
有选择题答案 → 可做规则 reward（这正是「从可验证任务开始」）。无复杂计算，却要理解动机与语境。

训练流程（沿用 R1-Zero 精神）

- 1 选 instruction model 作初始策略
- 2 Social IQa → 模型模板格式
- 3 模型对每题生成带 CoT 回答
- 4 规则 reward 检查格式 + 最终答案
- 5 REINFORCE++ 据 reward 更新
- 6 记录 reward/长度/语言混杂/CoT
- 7 数学 benchmark 上做 OOD 评估

在社会推理上做 RL，部分数学 OOD 任务正迁移

数学 CoT 测试准确率 (OOD)

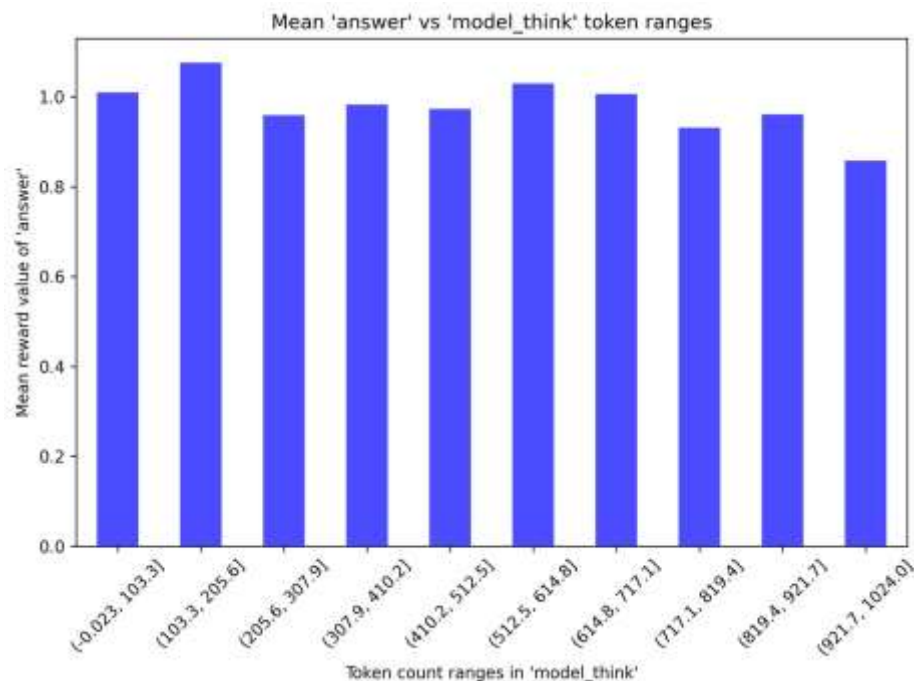


怎么读这张图

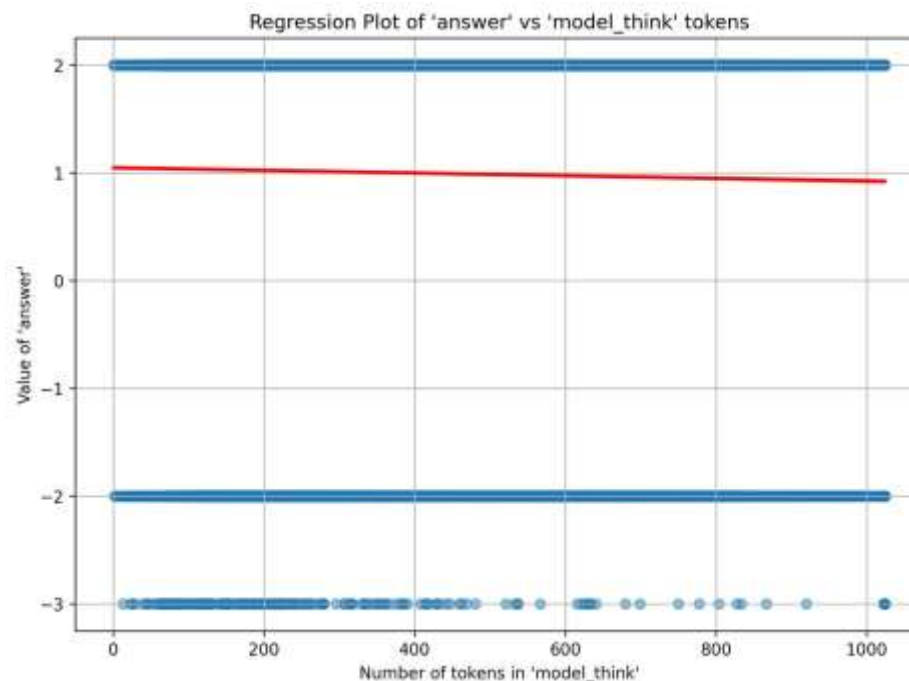
- **math-cot 大涨** (0.27→0.44)
- cmath-cot 小升; gsm8k 略降——并非全面提升
- RL 没只学社会题答案，而是改变了「面对推理问题的行为策略」：更愿展开推理、检查选项、守格式

谨慎：OOD 泛化需更多种子/模型/基准与统计分析——宜作为「有启发的现象」，呼应「评估 OOD 比看 reward 更重要」。

token 数量不是能力指标；overthinking 与难度有关



各 think-token 区间的平均 answer 奖励 ≈ 持平，不随更长推理上升。



answer 值 vs think token 数的回归线几乎水平/微降。

结论：更长 CoT 不稳定出现，且不等于更高 EQ/准确率；overthinking 多在「难度适中」时才有收益 → 关注 curriculum。与 agent 一致：走更多步/调更多工具 ≠ 更有效。

语言混杂存在但不普遍；对 Agentic RL 的启发

thinking 语言分布

类别	占比
Only English	98.23%
Only Chinese	0.00%
Mixed EN+ZH	1.75%

混杂存在但不高 → 只看 accuracy/reward 会错过行为层变化，训练日志分析重要。

对 Agentic RL 的五点启发

- ① 纯 RL 复现从可验证任务开始 (Social IQa 有选择题答案)
- ② 小模型实验也有价值 (3B 成本低，快速验证 reward/数据/算法/日志)
- ③ 不要只看 reward (同看 OOD、长度、格式错误率、语言、样例)
- ④ 长 CoT / 长轨迹都不是目标 (目标是成功率、泛化、可靠)
- ⑤ R1-Zero 的内核：用可验证反馈让模型自己探索策略——可迁移到工具调用与更复杂 agent

SUMMARY

总结

- **核心**：Agentic RL 不是把 PPO/GRPO 套到 LLM，而是重新定义训练对象——从「回答分布」到「能在环境中行动、观察、修正、完成任务的策略」。
- **能力 × 任务 × 工程**：塑造规划/工具/记忆/自改/推理/感知；进入搜索/代码/数学/GUI/视觉/多智能体；真正难的常是环境、奖励、轨迹收集与评估。
- **DeepEnlighten 提醒**：RL 后训练最有价值的不是让模型输出更长，而是在可验证反馈下形成更有效、更可泛化的行为策略。

给团队的四条实践

① 从可验证任务起步 ② 先做强评估再做强训练 ③ 保留轨迹做失败分析 ④ 小心把 benchmark 训成游戏

一句话：Agentic RL 把大模型从「会说答案」推向「会完成任务」——关键不只是更强模型，也包括更好的环境、更可靠的奖励、更扎实的行为评估。